Logistic regression cost function

$$J(\theta) = \frac{1}{m} \sum_{i=1}^{m} \text{Cost}(h_\theta(x^{(i)}), y^{(i)})$$

$$\text{Cost}(h_\theta(x), y) = \begin{cases} -\log(h_\theta(x)) & \text{if } y = 1 \\ -\log(1 - h_\theta(x)) & \text{if } y = 0 \end{cases}$$

Note: $y = 0$ or 1 always

$$\text{Cost}(h_\theta(x), y) = -y \log(h_\theta(x)) - (1 - y) \log(1 - h_\theta(x))$$

$$h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Logistic regression cost function

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m} \text{Cost}(h_\theta(x^{(i)}), y^{(i)})$$

$$= -\frac{1}{m}[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_\theta(x^{(i)}))]$$

$P(y=1|x)$ $P(y=0|x)$

Cross entropy تابع خطای

Gradient Descent

θ

$$J(\theta) = -\frac{1}{m}[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_\theta(x^{(i)}))]$$

Want $\min_\theta J(\theta)$:

Repeat $\{$

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

$\}$

(simultaneously update all θ_j)

$j = 0 \ldots n$

$$\frac{\partial}{\partial \theta_j}\sigma(\theta^T x)?$$

$\sigma'(z) = \sigma(z)(1-\sigma(z))$

$$\frac{\partial}{\partial z}\sigma(z) = \sigma(z)[1 - \sigma(z)]$$

True fact about sigmoid functions

.

$$\frac{\partial}{\partial \theta_j}\sigma(\theta^T x) = \frac{\partial}{\partial z}\sigma(z) \cdot \frac{\partial z}{\partial \theta_j}$$

Chain rule!

$$\frac{\partial}{\partial \theta_j}\sigma(\theta^T x) = \sigma(\theta^T x)[1 - \sigma(\theta^T x)]x_j$$

Plug and chug

Gradient Descent

$$J(\theta) = -\frac{1}{m}[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_\theta(x^{(i)}))]$$

Want $\min_\theta J(\theta)$: θ_{init}

Repeat $\{$

$$\theta_j := \theta_j - \alpha \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

$= \sigma(\theta^T x)$

(simultaneously update all θ_j) $j = 0 \ldots n$

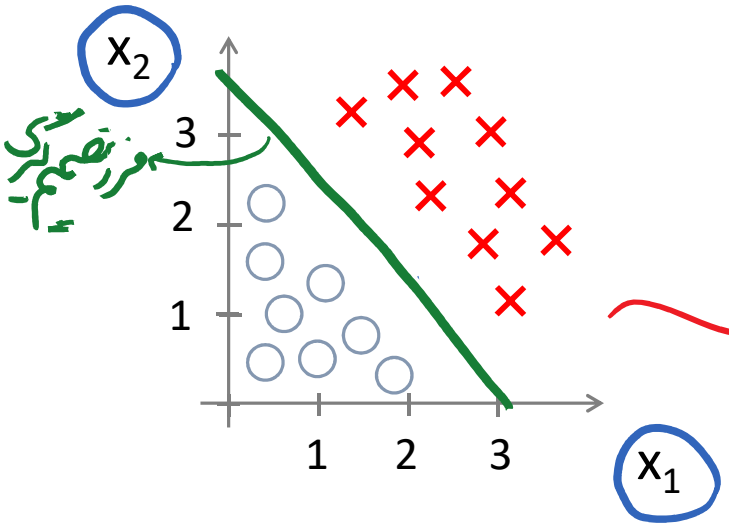
$\}$

$$\theta = \begin{bmatrix} \theta_0 \\ \vdots \\ \theta_n \end{bmatrix}$$

Algorithm looks identical to linear regression!

Decision Boundary

$\theta^T x = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2$, $x_0 = 1$



مرز تصمیم گیری

$$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

$(x^{(i)}, y^{(i)})$ $i = 1 \dots m$ $\Rightarrow$ $\theta_0 = -3$ $\theta_1 = \theta_2 = 1$

Predict "$y = 1$" if $-3 + x_1 + x_2 \geq 0$ $\rightarrow$ $y = 1$

$\theta^T x \geq 0$ $\quad < 0$ $\rightarrow$ $y = 0$

مرز تصمیم گیری

$th = 0.5$

$\sigma(\theta^T x)$ $\theta_0, \theta_1, \theta_2$

Non-linear decision boundaries

$h_\theta(x) = \sigma(p_\theta(x))$

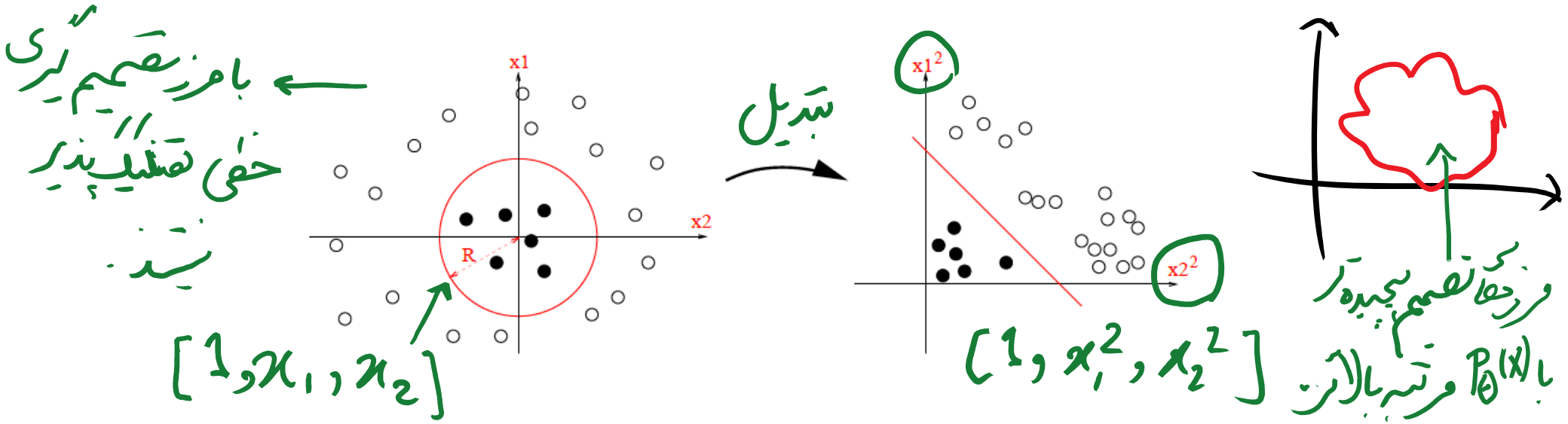
$p_\theta(x) = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1 x_2 + \theta_4 x_1^2 + \theta_5 x_2^2$

Transform input features in suitable way

$$\phi(x_1, x_2) = [1, x_1^2, x_2^2] \quad x_1, x_2, x_1, x_2$$

$w = [-R^2, 1, 1]$. Then $w^T \phi(x) = x_1^2 + x_2^2 - R^2$ $\geq 0 \rightarrow y=1$ $< 0 \rightarrow y=0$

Decision boundary (where f(x) = 0) defines a circle with radius R



logistic regression (Probabilistic view)

$\underline{w}^T \underline{x} + b \equiv \theta^T x$

$\theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n$

b $\quad$ $\underline{w}^T \underline{x}$

Binary logistic regression often follows the following model

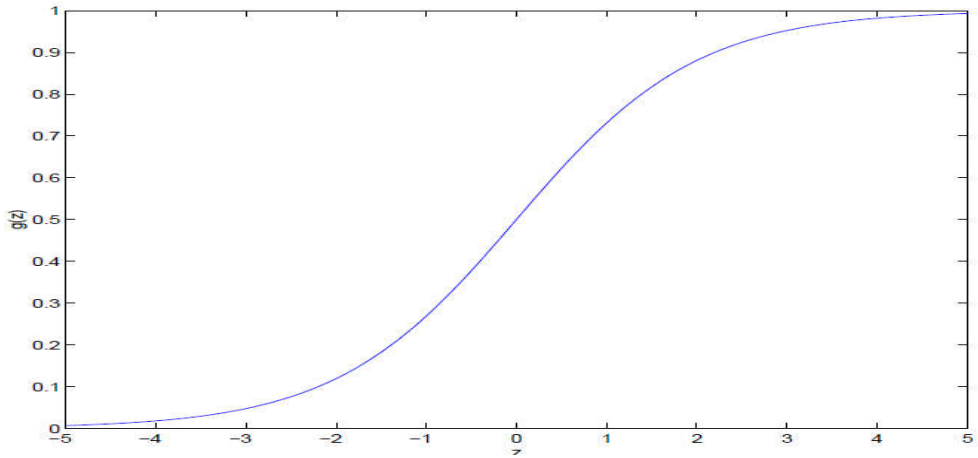
$$p(y|x;\theta) = \mathrm{Ber}(y|\sigma(\underline{w}^T \underline{x} + b))$$

Bernoulli: $p = \sigma(\theta^T x)$

Bernoulli — sigmoid — weight — bias

$$f(k;p) = \begin{cases} p & \text{if } k = 1, \\ q = 1 - p & \text{if } k = 0. \end{cases}$$

$$p(y = 1|x;\theta) = \sigma(a) = \frac{1}{1 + e^{-a}}, \text{where } a = log\frac{p}{1-p}$$



logistic regression (Probabilistic view)

Finding the Maximum Likelihood (ML) solution is equivalent to minimizing the cross entropy cost function

$P(y \mid x; \theta)$ $\quad P_\theta(y \mid x)$

$\max_\theta \log P_\theta(y \mid x)$ $\equiv$ $\min_\theta J(\theta)$

log-likelihood $\quad$ Cross entropy

$P_\theta(y=1 \mid x) = h_\theta(x) = \sigma(\theta^T x)$

$P_\theta(y=0 \mid x) = 1 - h_\theta(x) = 1 - \sigma(\theta^T x)$

log_likelihood:

$$P_\theta(y^{(i)} \mid x^{(i)}) = h_\theta(x^{(i)})^{y^{(i)}} \times (1-h_\theta(x^{(i)}))^{(1-y^{(i)})}$$

$$P_\theta(\underline{y} \mid \underline{\underline{X}}) = \prod_{i=1}^{m} \left(h_\theta(x^{(i)})^{y^{(i)}} \times (1-h_\theta(x^{(i)}))^{(1-y^{(i)})} \right)$$

Likelihood

$$\underline{y} = \begin{bmatrix} y_1 \\ y_m \end{bmatrix}$$

log_likelihood:

$$\log P_\theta(\underline{y} \mid \underline{\underline{X}}) = \sum_{i=1}^{m} \left[y^{(i)} \log h_\theta(x^{(i)}) + (1-y^{(i)}) \log(1-h_\theta(x^{(i)})) \right]$$

$$\max_\theta \log P_\theta(\underline{y} \mid \underline{\underline{X}}) = \min\left(-\sum_{i=1}^{m} y^{(i)} \dots \right)$$

cross_entropy 손실

اگربا فرض تصادفی بودن مشاهدات y که از توزیع برنولی باشند، تخمین ML برای پارامتر $\boldsymbol{\theta}$ را بدست آوریم، معادل است با $\boldsymbol{\theta}$ که با می نیمم کردن تابع هزینه cross entropyبدست می آوریم.

Multiclass classification

Email foldering/tagging: Work, Friends, Family, Hobby

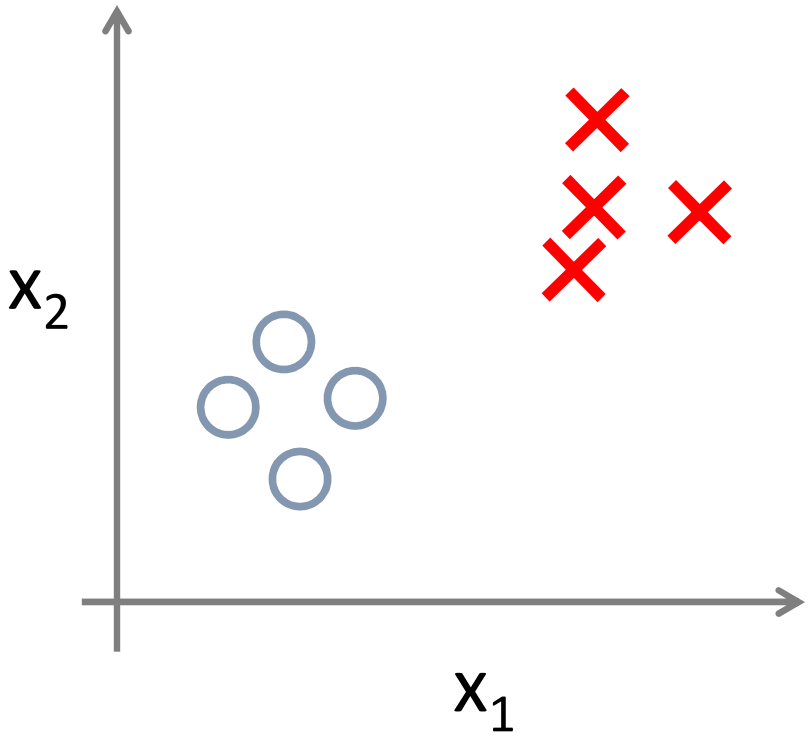
Medical diagrams: Not ill, Cold, Flu

Weather: Sunny, Cloudy, Rain, Snow

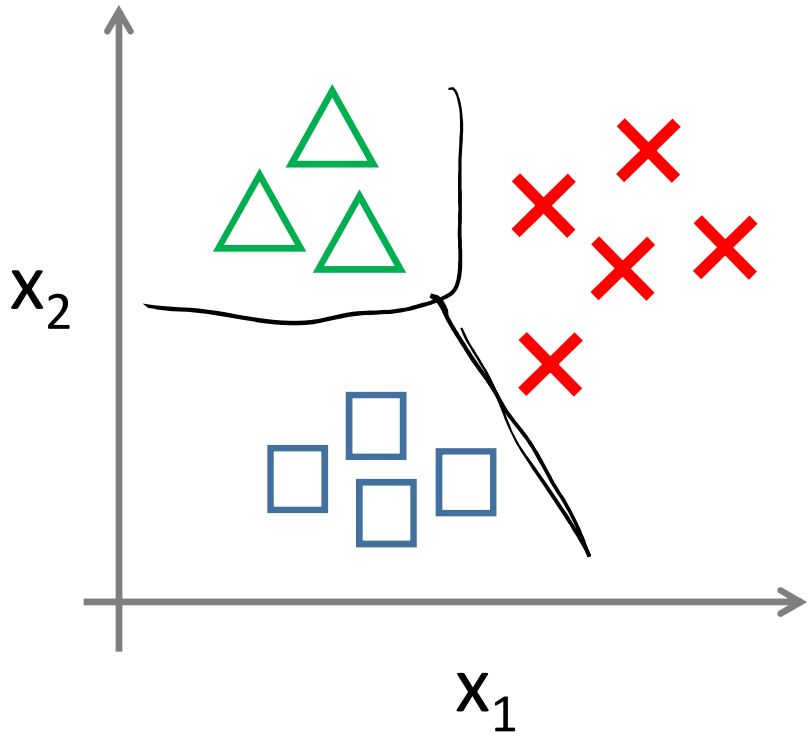
$y \in \{0, 1, 2, 3\}$

$x_{test} \rightsquigarrow$ کلاس 2

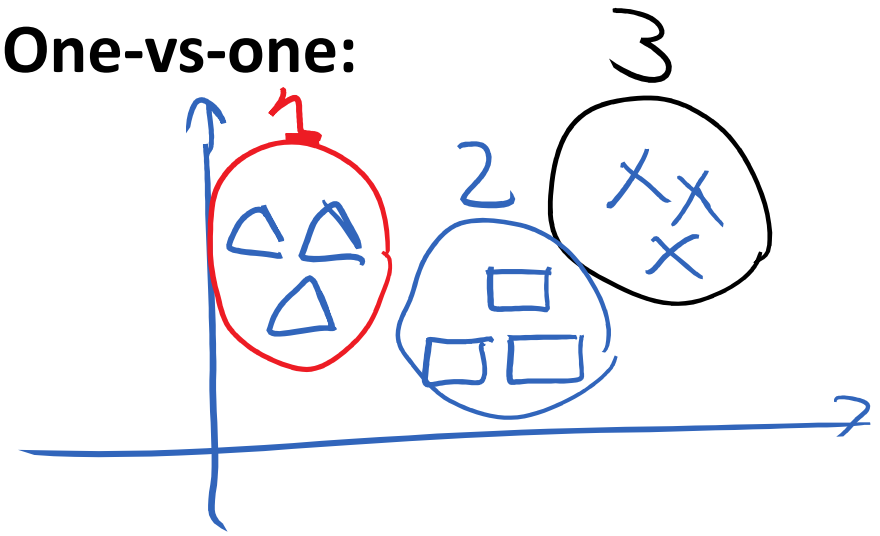
Binary classification:



Multi-class classification:



One-vs-one:



1 - 2

2 - 3

1 - 3

$k = 3$

$\binom{k}{2}$

تعداد حالت‌های ممکن

$x_{test} \longrightarrow \binom{k}{2}$ حالت

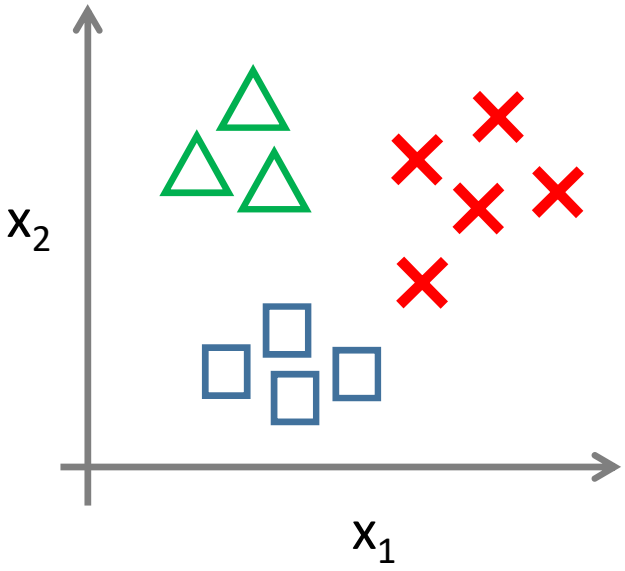
1

1

3

$\Rightarrow \quad y_{test} = 1$

One-vs-all:

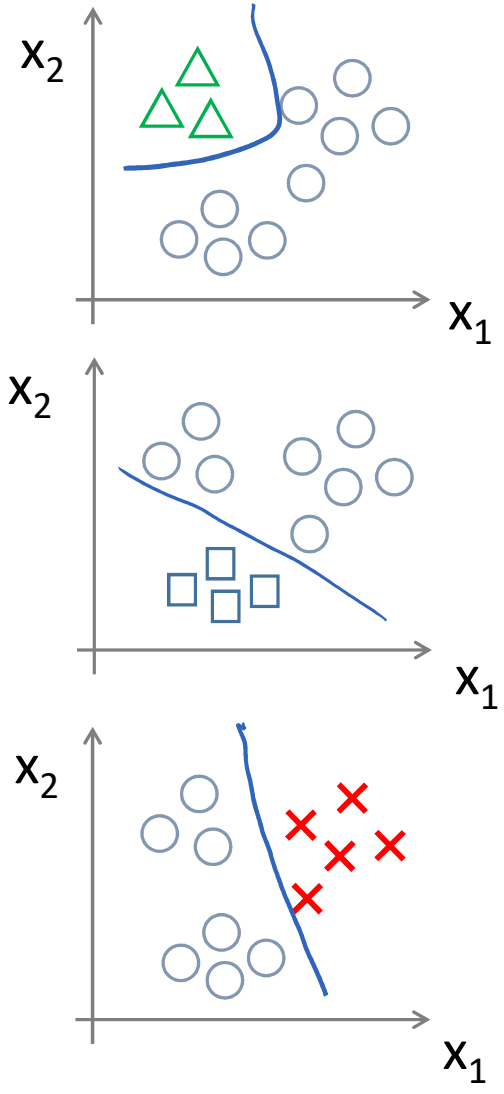


Class 1: △

Class 2: □

Class 3: ×

$$h_\theta^{(i)}(x) = P(y = i|x;\theta) \qquad (i = 1, 2, 3)$$



Softmax regression

سی تعمیم یافته logistic regression برای کلاس $k>2$

$(x^{(i)}, y^{(i)})$ $i=1..m$ $\quad y^{(i)} \in \{1,2,..,k\}$

MNIST $k=10$

$p(y=k \mid x) \Rightarrow h_\theta(x) = \begin{bmatrix} p(y=1 \mid x;\theta) \\ p(y=2 \mid x;\theta) \\ p(y=k \mid x;\theta) \end{bmatrix}$

$$h_\theta^{(j)}(x) = \frac{e^{\theta^{(j)T} x}}{\sum_{i=1}^{k} e^{\theta^{(i)T} x}}$$

$$\sum_{j=1}^{k} h_\theta^{(j)}(x) = 1$$

$\theta = [\theta^{(1)}, \dots, \theta^{(k)}]$ $(n+1) \times k$: پارامتر θ

$\theta^{(k)} = \theta^{(1)} - \theta^{(2)} - \dots - \theta^{(k-1)}$ $\quad (n+1)\times 1$

Cross entropy cost function:

1 (عبارت درست) $= 1$ $\quad$ 1 (عبارت نادرست) $= 0$

$$J(\theta) = -\left[\sum_{i=1}^{m}\sum_{j=1}^{K} 1\{y^{(i)}=j\} \log \frac{\exp(\theta^{(j)T} x^{(i)})}{\sum_{k=1}^{K}\exp(\theta^{(k)T} x^{(i)})}\right]$$

تعمیم یافته تابع هزینه Log. Reg.

$K=4$

$x_{t_3+} \rightarrow h_\theta(x_{t_3+}) = \begin{bmatrix} 0.8 \\ 0.1 \\ \%5 \\ 0/05 \end{bmatrix} \rightarrow y_{t_3+} = 1$

$P(y|x,\theta) \sim \text{Categorical}(P_1, P_2, \dots, P_k)$

$P_k = 1 - \sum_{i=1}^{k-1} P_i$

Regularization

Regularization

A fundamental problem is that the algorithm tries to pick parameters that minimize loss on the training set, but this may not result in a model that has low loss on future data. This is called **overfitting**.

example

suppose we want to predict the probability of heads when tossing a coin. We toss it $N = 3$ times and observe 3 heads. The MLE is $\theta_{\text{mle}} = N_1/(N_0 + N1) = 3/(3 + 0) = 1$. However, if we use $\text{Ber}\left(y \mid \theta_{\text{mle}}\right)$ as our model, we will predict that all future coin tosses will also be heads, which seems rather unlikely.

Regularization

- The core of the problem is that the model has enough parameters to perfectly fit the observed training data. so it can **perfectly match** the empirical distribution.
- However, in most cases the empirical distribution is not the same as the true distribution, so putting all the probability mass on the observed set of N examples will not leave over any probability for novel data in the future. That is, **the model may not generalize**.

Solution

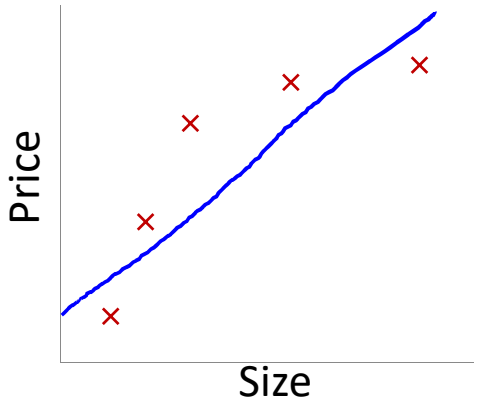
The main solution to overfitting is to use regularization, which means to add a penalty term to the Cost function. Thus we optimize an objective of the form

$$\mathcal{L}(\boldsymbol{\theta}; \lambda) = \left[\frac{1}{M}\sum_{i=1} \ell(\mathbf{y}_i, \boldsymbol{\theta}; \mathbf{x}_i)\right] + \lambda C(\boldsymbol{\theta})$$

$\lambda \geq 0$ is a tuning parameter and control the relative impact of these two terms on the regression coefficient estimates.

When $\lambda = 0$, the penalty term has no effect
However, as $\lambda \to \infty$, the impact of the shrinkage penalty grows, and the ridge regression coefficient estimates will approach zero.

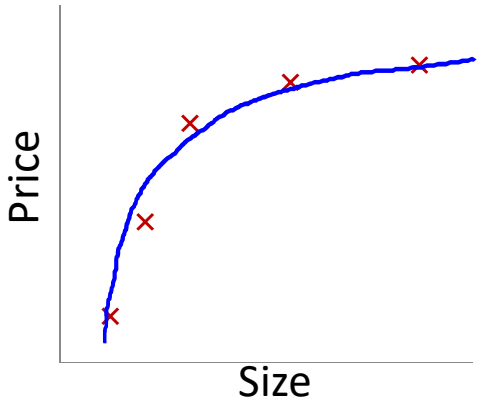
Example: Linear regression (housing prices)



$\rightarrow \theta_0 + \theta_1 x$

"Underfit" "High bias"

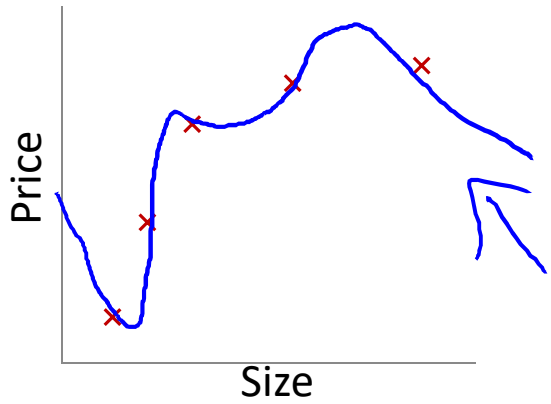
λ ↑



$\rightarrow \theta_0 + \theta_1 x + \theta_2 x^2$

"Just right"

λ



$\rightarrow \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

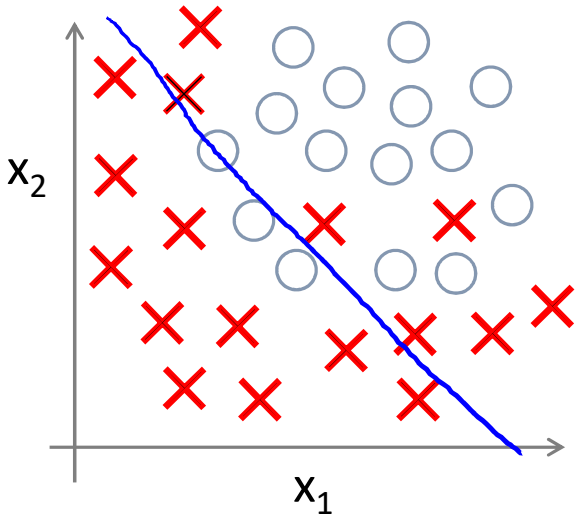
"Overfit" "High variance"

$\lambda = 0$

Overfitting: If we have too many features, the learned hypothesis may fit the training set very well ($J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2 \approx 0$), but fail to generalize to new examples (predict prices on new examples).

Example: Logistic regression

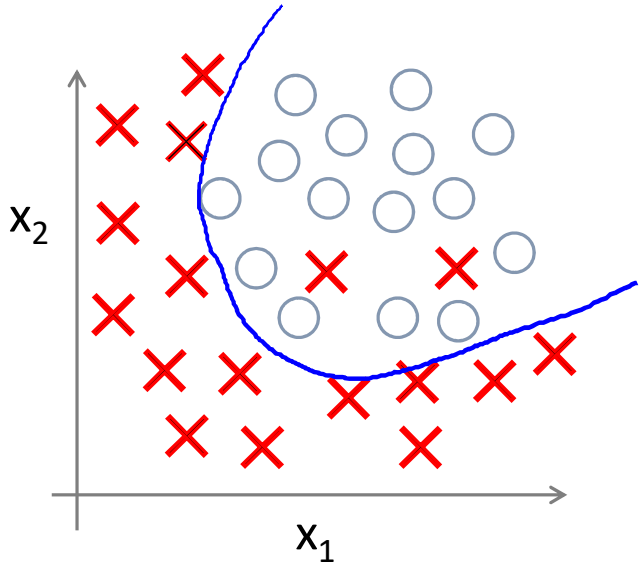
(طبقه‌بندی)



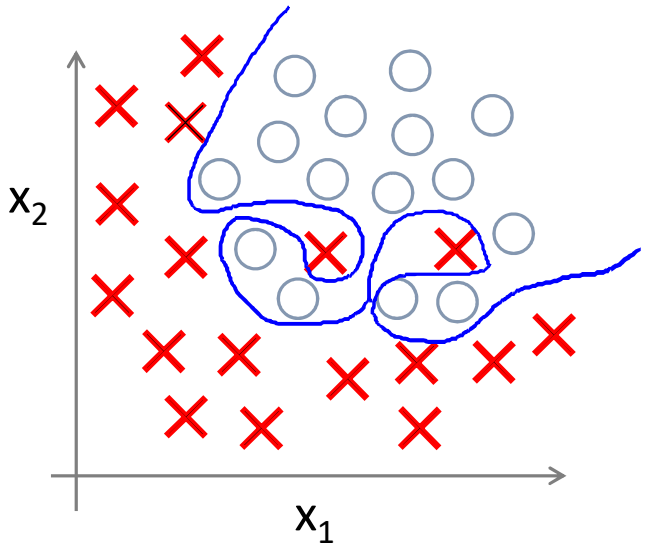
$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$

(g = sigmoid function)

"Underfit"



$g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2)$

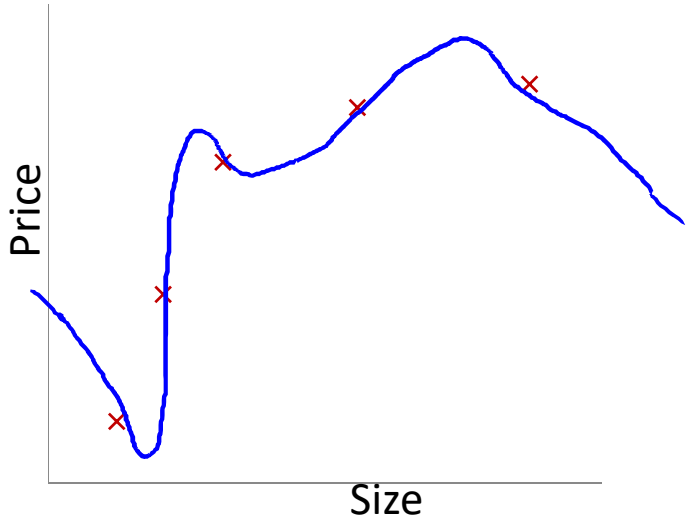


$g(\theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^2 x_2 + \theta_4 x_1^2 x_2^2 + \theta_5 x_1^2 x_2^3 + \theta_6 x_1^3 x_2 + \dots)$

"Overfit"

Addressing overfitting:

$x_1 =$ size of house
$x_2 =$ no. of bedrooms
$x_3 =$ no. of floors
$x_4 =$ age of house
$x_5 =$ average income in neighborhood
$x_6 =$ kitchen size
$\vdots$
x_{100}



Addressing overfitting:

Options:

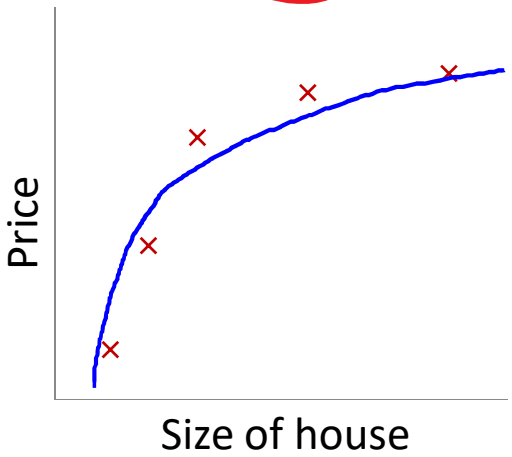
1. Reduce number of features
   - Manually select which features to keep.
   - Model selection algorithm (later in course).
2. Regularization
   - Keep all the features, but reduce magnitude/values of parameters θ_j
   - Works well when we have a lot of features, each of which contributes a bit to predicting y.

Regularization

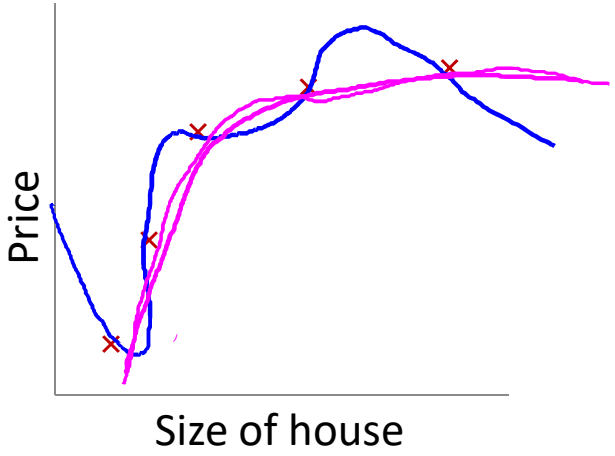
Cost function

Intuition

Bias-Variance tradeoff



$\theta_0 + \theta_1 x + \theta_2 x^2$



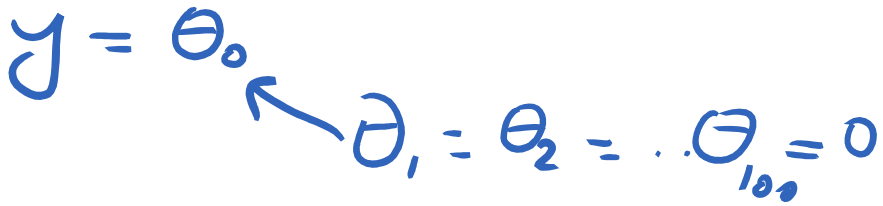
$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

Suppose we penalize and make θ_3, θ_4 really small.

$$\min_\theta \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2 + 1000\,\theta_3^2 + 1000\,\theta_4^2$$

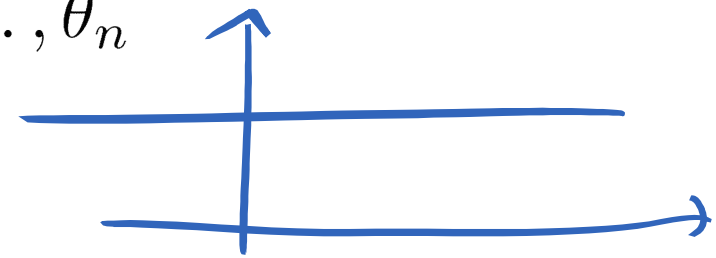
$\theta_3 \approx 0 \qquad \theta_4 \approx 0$

Regularization.



Small values for parameters $\theta_0, \theta_1, \ldots, \theta_n$

- "Simpler" hypothesis
- Less prone to overfitting



Housing:

- Features: $x_1, x_2, \ldots, x_{100}$
- Parameters: $\theta_0, \theta_1, \theta_2, \ldots, \theta_{100}$

$$J(\theta) = \frac{1}{2m}\left[\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^{n} \theta_j^2\right]$$

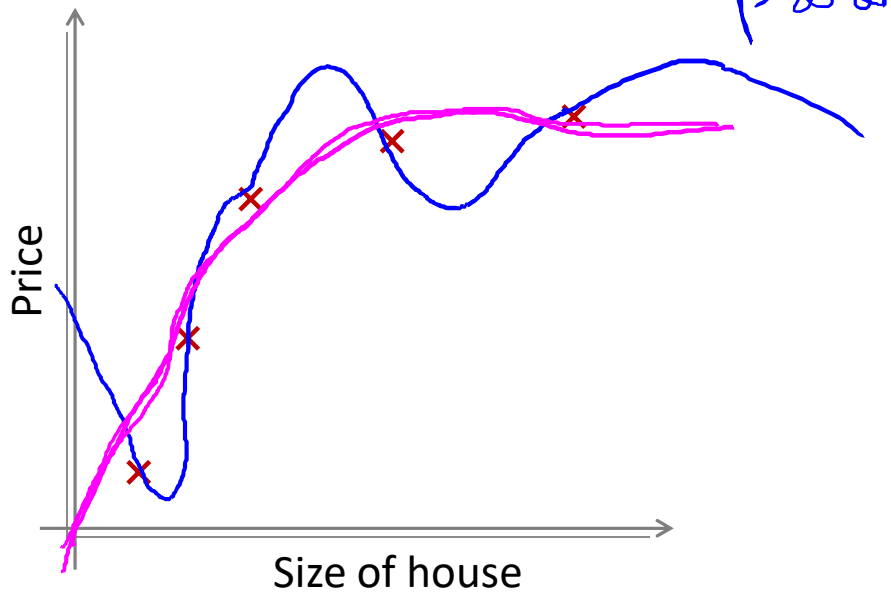
$\theta_1, \theta_2, \theta_3, \ldots, \theta_{100}$

Regularization.

$$J(\theta) = \frac{1}{2m}\left[\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^{n}\theta_j^2\right]$$

regularization parameter

$$\min_\theta J(\theta)$$



L1_norm: $\lambda \sum_{j=1}^{n} |\theta_j|$

In regularized linear regression, we choose θ to minimize

$$J(\theta) = \frac{1}{2m}\left[\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^{n} \theta_j^2\right]$$

L2_norm regularization

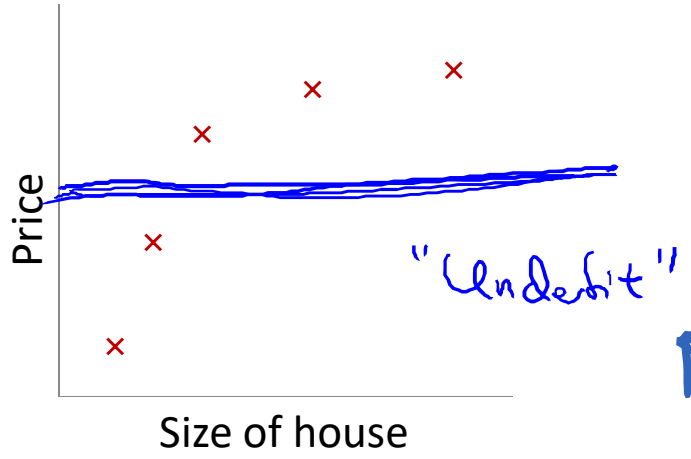
What if λ is set to an extremely large value (perhaps for too large for our problem, say $\lambda = 10^{10}$)?

- Algorithm works fine; setting λ to be very large can't hurt it
- Algortihm fails to eliminate overfitting.
- Algorithm results in underfitting. (Fails to fit even training data well).
- Gradient descent will fail to converge.

In regularized linear regression, we choose θ to minimize

$$J(\theta) = \frac{1}{2m}\left[\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \lambda\sum_{j=1}^{n}\theta_j^2\right]$$

What if λ is set to an extremely large value (perhaps for too large for our problem, say $\lambda = 10^{10}$)?



$h_\theta(x)$

$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

$\theta_1, \theta_2, \theta_3, \theta_4$

$\theta_1 \approx 0, \theta_2 \approx 0$

$\theta_3 \approx 0, \theta_4 \approx 0$

$h_\theta(x) = \theta_0$